MLOps Assignment 2

Hugging Face Fine-Tuning, Experiment Tracking & Model Deployment

PGD AI Program | IIT Jodhpur | MLOps

Student ID: G25AIT2134

Resource	Link
Hugging Face Model	huggingface.co/G25AIT2134/distilbert-goodreads-genres
W&B Dashboard	wandb.ai/g25ait2134-iitj/mlops-assignment2/runs/5s2gs1m0
GitHub Repository	github.com/g25ait2134-tech/MLOpsAssignment2
Kaggle Notebook	kaggle.com/code/shyamg25ait2134/notebook828cbfd1f6

Final Results at a Glance

Accuracy	F1 Score (weighted)	Eval Loss	Training Time
0.59	**0.585**	**2.297**	**~628 seconds**

1. Model Selection Rationale

For this assignment, **distilbert/distilbert-base-uncased** was selected from the Hugging Face Hub. DistilBERT is a distilled version of the original BERT model, trained using knowledge distillation to retain 97% of BERT's language understanding while being 40% smaller and 60% faster at inference. These properties make it highly practical for an MLOps context where training efficiency, deployment size, and iteration speed matter as much as raw accuracy.

The **uncased** variant was chosen deliberately over the cased version. The classification task — predicting book genres from Goodreads reviews — relies on the semantic content of words such as 'romance', 'mystery', or 'fantasy', not on whether they appear capitalised. Uncased models normalise all text to lowercase before tokenisation, reducing vocabulary size and improving generalisation across the varied capitalisation styles found in user-generated review text. A cased model would treat 'Fantasy' and 'fantasy' as distinct tokens, adding noise without adding signal.

DistilBERT's architecture — 6 transformer layers, 768 hidden dimensions, 12 attention heads — is well-suited to sequence classification tasks of this type. Its pre-training on a large English corpus (BookCorpus and English Wikipedia) gives it strong priors on the kind of literary language that appears in book reviews, making fine-tuning on the UCSD Goodreads dataset efficient and effective.

2. Kaggle Training Setup

2.1 Platform Configuration

Training was performed on a **Kaggle Notebook** with a free-tier **GPU T4** accelerator enabled under Settings → Accelerator. Internet access was enabled (Settings → Internet toggle ON) to allow package installation, dataset streaming from the UCSD servers, and pushing the trained model to Hugging Face Hub.

Setting	Value
Accelerator	GPU T4 x2
Internet	Enabled
Kaggle Secrets	WANDB_API_KEY, HF_TOKEN
Framework	PyTorch + Hugging Face Transformers
Training time	~628 seconds (~10 minutes)

2.2 Kaggle Secrets

API tokens were never hardcoded. They were stored securely using Kaggle's built-in Secrets manager (Add-ons → Secrets) and accessed at runtime:

```
from kaggle_secrets import UserSecretsClient
secrets = UserSecretsClient()
WANDB_API_KEY = secrets.get_secret('WANDB_API_KEY')
HF_TOKEN      = secrets.get_secret('HF_TOKEN')
import os
os.environ['WANDB_API_KEY'] = WANDB_API_KEY
os.environ['HF_TOKEN']      = HF_TOKEN
```

2.3 Creating a W&B; API Key

To obtain a Weights & Biases API key:

- Go to **wandb.ai** and create a free account.

- Navigate to **Settings** → **API Keys** (wandb.ai/settings).
- Click **New Key**, copy the generated key.
- In Kaggle: Add-ons → Secrets → Add new secret. Name: **WANDB_API_KEY**, Value: paste your key.
- Check 'Attach to notebook' and save.

2.4 Creating a Hugging Face Token

To obtain a Hugging Face access token:

- Go to **huggingface.co** and create a free account.
- Navigate to **Settings** → **Access Tokens** (huggingface.co/settings/tokens).
- Click **New token**, choose role **Write** (needed to push models), and generate.
- Copy the token (starts with **hf_...**).
- In Kaggle: Add-ons → Secrets → Add new secret. Name: **HF_TOKEN**, Value: paste your token.

2.5 Training Hyperparameters

Hyperparameter	Value
Model	distilbert/distilbert-base-uncased
Epochs	3
Train batch size	16 per device
Eval batch size	32 per device
Learning rate	3e-5
Warmup steps	100
Weight decay	0.01
Logging steps	every 50 steps
Eval strategy	every epoch
Save strategy	every epoch
Best model loaded	Yes (load_best_model_at_end=True)
W&B tracking	report_to=wandb

3. Experiment Tracking with W&B

Weights & Biases was integrated via the Hugging Face Trainer's **report_to="wandb"** argument. This single parameter caused the Trainer to automatically log training loss (every 50 steps), validation loss (every epoch), accuracy, F1 score, learning rate schedule, and GPU utilisation —

without any manual logging code in the training loop.

In addition, a **wandb.init()** call at the start of training logged all hyperparameters as a config object, making every run fully reproducible. A 'platform' tag was added to distinguish Kaggle runs from any local runs:

```
wandb.init(
    project='mlops-assignment2',
    name='distilbert-run-1',
    config={
        'model': model_name,  'epochs': 3,
        'batch_size': 16,     'learning_rate': 3e-5,
        'max_length': 512,    'dataset': 'UCSD Goodreads',
        'platform': 'Kaggle'
    }
)
```

After training, final metrics were explicitly logged to W&B and the full per-class classification report was uploaded as a **versioned W&B Artifact**, providing a permanent record tied to the specific run:

```
wandb.log({'final/loss': 2.297, 'final/accuracy': 0.59, 'final/f1': 0.585})
artifact = wandb.Artifact('eval-report', type='evaluation')
artifact.add_file('eval_report.json')
wandb.log_artifact(artifact)
```

W&B Metric Logged	Value
final/accuracy	0.59
final/f1 (weighted)	0.585
final/loss (eval)	2.297
train/loss (final)	1.712
train samples/sec	37.5
Runtime	628 seconds
Global steps	600

4. Evaluation Results

The model was evaluated on a held-out test set of **1,600 reviews** (200 per genre). The overall accuracy of **59%** is a meaningful result for an 8-class classification task, where random-chance performance would be 12.5%. The TF-IDF logistic regression baseline run prior to BERT fine-tuning gives useful context for interpreting this number.

Performance varied considerably across genres, reflecting genuine differences in how distinctively each genre's reviews are written:

Genre	Precision	Recall	F1-Score

children	0.632	0.585	0.608
comics & graphic	0.761	0.810	0.785
fantasy & paranormal	0.430	0.490	0.458
history & biography	0.598	0.550	0.573
mystery / thriller	0.490	0.605	0.541
poetry	0.741	0.815	0.776
romance	0.613	0.585	0.598
young adult	0.424	0.280	0.337
Weighted average	**0.586**	**0.590**	**0.585**

Per-class results from eval_report.json (test set, 200 samples per genre)

4.1 Interpretation

Best-performing genres — Poetry (F1: 0.776) and Comics & Graphic (F1: 0.785): These genres have highly distinctive vocabulary. Poetry reviews use terms like 'verse', 'stanza', 'lyrical'; comics reviews mention 'panels', 'artwork', 'illustrations'. This makes them easier for the model to separate from the rest.

Most confused genres — Young Adult (F1: 0.337) and Fantasy & Paranormal (F1: 0.458): Young Adult fiction spans many genres (fantasy, romance, mystery), so its reviews share vocabulary with multiple other categories. Fantasy and YA in particular share themes of magic, coming-of-age, and adventure, causing frequent cross-class confusion.

Overall weighted F1 of 0.585 reflects the balanced nature of the dataset (200 samples per class) and is a reasonable result for a 3-epoch fine-tune on a small sample (1,000 reviews per genre from a much larger corpus).

5. Model Publishing — Hugging Face Hub

The fine-tuned model and tokenizer were pushed to Hugging Face Hub from within the Kaggle Notebook, making the model publicly accessible for inference by anyone with a Hugging Face account:

```
from huggingface_hub import login
login(token=HF_TOKEN)

repo_name = 'G25AIT2134/distilbert-goodreads-genres'
model.push_to_hub(repo_name)
tokenizer.push_to_hub(repo_name)

wandb.run.summary['huggingface_model'] = \
    f'https://huggingface.co/{repo_name}'
```

The model is publicly accessible at: **huggingface.co/G25AIT2134/distilbert-goodreads-genres**. It can be loaded directly via the Hugging Face pipeline API for inference:

```
from transformers import pipeline
classifier = pipeline('text-classification',
                      model='G25AIT2134/distilbert-goodreads-genres')
classifier('A gripping tale of dragons and ancient kingdoms...')
# => [{'label': 'fantasy_paranormal', 'score': 0.82}]
```

The **inference.py** script in the GitHub repository provides a ready-to-run wrapper around this pipeline for local use.

6. Challenges & Learnings

6.1 Challenges faced

- **Dataset streaming on Kaggle:** The UCSD Goodreads dataset is served via HTTP as gzipped JSON. Streaming it with requests inside a Kaggle environment required Internet to be toggled on explicitly in Session Options — a non-obvious step that caused initial failures.
- **Label ordering non-determinism:** Using Python's set() to build the label-to-ID mapping produces a different ordering each run, making results non-reproducible. This was fixed by switching to sorted(set(train_labels)), ensuring consistent label IDs across runs.
- **wandb.finish() placement:** Calling wandb.finish() before the analysis and visualisation cells at the bottom of the notebook caused subsequent W&B summary updates to fail silently. Moving it to the final cell resolved this.
- **Cased vs uncased model choice:** The starter notebook used distilbert-base-cased. Recognising that genre classification doesn't benefit from capitalisation awareness, switching to the uncased variant improves generalisation with no added cost.

6.2 What would be done differently

- **Larger sample size:** Only 1,000 reviews per genre were used. Increasing this to 5,000+ would likely raise accuracy substantially, at the cost of longer training time.
- **Validation split:** A proper train/validation/test split (rather than just train/test) would allow hyperparameter tuning without touching the test set, giving a more reliable estimate of generalisation performance.
- **Multiple W&B runs:** Running the training with different learning rates or batch sizes and comparing them in the W&B dashboard would demonstrate the experiment-tracking workflow more fully.
- **Model card on Hugging Face:** Adding a README/model card to the HF repository with intended use, training details, and limitations would follow best MLOps practices.

6.3 Key learnings

This assignment gave hands-on experience with the core MLOps loop: train → track → publish. The main takeaway is that the tooling around a model (W&B for reproducibility, Hugging Face Hub for

distribution, Kaggle Secrets for secure credential management) is as important as the model itself. Every decision — from label ordering to wandb.finish() placement — has a downstream impact on reproducibility and correctness.